Object Oriented Programming (CSEG1044)

Unit 03 – Lecture 04: Java I/O Streams + FileReader/FileWriter

April 1, 2026

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Nested Classes, Exceptions, Multithreading, and I/O

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Nested Classes, Exceptions, Multithreading, and I/O
- **Course thread:** Use Java I/O Streams + FileReader/FileWriter to make Campus Resource Manager safer, more modular, and easier to recover when something goes wrong.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Recall Before You Start

- **Exceptions**: File work is one of the most common places where exceptions matter. Main reference: `Unit 03 / Lecture 02 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Recall Before You Start

- **Exceptions**: File work is one of the most common places where exceptions matter. Main reference: `Unit 03 / Lecture 02 slides/notes`.
- **Strings and loops**: Reading and writing files still depends on iterating and representing text correctly. Main reference: `Unit 01 / Lecture 04 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Learning Outcomes

- Explain stream model; choose byte vs character streams.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Learning Outcomes

- Explain stream model; choose byte vs character streams.
- Read/write text files with `FileReader`/`FileWriter` and buffering.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Learning Outcomes

- Explain stream model; choose byte vs character streams.
- Read/write text files with `FileReader`/`FileWriter` and buffering.
- Use `try-with-resources` for exception-safe I/O code.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Terms to Know

- **Stream**: sequence of data flowing from a source to a destination.
- **Input stream** vs **output stream**: read vs write.
- **Byte stream**: reads/writes raw bytes (`InputStream`/`OutputStream`).
- **Character stream**: reads/writes characters (`Reader`/`Writer`).
- **Buffering**: collecting data in memory to reduce expensive I/O operations.
- **try-with-resources**: `try(...)` that auto-closes resources.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Stream model (big picture)* $\rightarrow$ *Byte streams vs character streams* $\rightarrow$ *Text file I/O with* `FileReader/FileWriter` $\rightarrow$ *Reading patterns (three common styles)*.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Stream model (big picture)* → *Byte streams vs character streams* → *Text file I/O with* `FileReader/FileWriter` → *Reading patterns (three common styles)*.
3. Trace the worked example and connect each line back to one concept frame.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Stream model (big picture)* → *Byte streams vs character streams* → *Text file I/O with* `FileReader/FileWriter` → *Reading patterns (three common styles)*.
3. Trace the worked example and connect each line back to one concept frame.
4. Attempt the mini exercises before moving to the recap and exit check.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Unit 03: Nested Classes, Exceptions, Multithreading & IO Streams

- Lecture focus: Java I/O Streams + FileReader/FileWriter

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Today: Roadmap

- Stream model; byte streams vs character streams

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Today: Roadmap

- Stream model; byte streams vs character streams
- File I/O with ‘FileReader‘ / ‘FileWriter‘ (+ buffering)

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Today: Roadmap

- Stream model; byte streams vs character streams
- File I/O with ‘FileReader‘ / ‘FileWriter‘ (+ buffering)
- Exception-safe code via ‘try-with-resources‘ and good error messages

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Stream model (big picture)

- Stream = sequence of data flowing from source to destination.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Stream model (big picture)

- Stream = sequence of data flowing from source to destination.
- Input stream reads; output stream writes.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Stream model (big picture)

- Stream = sequence of data flowing from source to destination.
- Input stream reads; output stream writes.
- Streams can be chained (file stream wrapped with buffer).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Byte vs character streams

- Byte streams: `InputStream`/`OutputStream` (binary: images, PDFs).

Rule of thumb

Use `Reader`/`Writer` for text; use `InputStream`/`OutputStream` for binary.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Byte vs character streams

- Byte streams: `InputStream`/`OutputStream` (binary: images, PDFs).
- Character streams: `Reader`/`Writer` (text files).

Rule of thumb

Use `Reader`/`Writer` for text; use `InputStream`/`OutputStream` for binary.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Text file I/O: `FileReader/FileWriter`

- `FileReader`: reads characters from a file.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Text file I/O: `FileReader`/`FileWriter`

- `FileReader`: reads characters from a file.
- `FileWriter`: writes characters to a file.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Text file I/O: `FileReader/FileWriter`

- `FileReader`: reads characters from a file.
- `FileWriter`: writes characters to a file.
- Wrap with buffering for performance and convenience.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Buffering: why it matters

- Disk I/O is slow; reading/writing character-by-character is inefficient.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Buffering: why it matters

- Disk I/O is slow; reading/writing character-by-character is inefficient.
- `BufferedReader` reads chunks and gives `readLine()`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Buffering: why it matters

- Disk I/O is slow; reading/writing character-by-character is inefficient.
- `BufferedReader` reads chunks and gives `readLine()`.
- `BufferedWriter` writes efficiently and provides `newLine()`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

`try-with-resources` (safe cleanup)

- Ensures files are closed even if an exception occurs.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

`try-with-resources` (safe cleanup)

- Ensures files are closed even if an exception occurs.
- Syntax: `try (BufferedReader br = ...){...}`

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

`try-with-resources` (safe cleanup)

- Ensures files are closed even if an exception occurs.
- Syntax: `try (BufferedReader br = ...){...}`
- Cleaner than manual `close()` in `finally`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Common beginner issue: file not found

- Paths are relative to the working directory (where you run `java`).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Common beginner issue: file not found

- Paths are relative to the working directory (where you run `java`).
- Check spelling, extension, and location.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Common beginner issue: file not found

- Paths are relative to the working directory (where you run `java`).
- Check spelling, extension, and location.
- Use absolute path if needed during debugging.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Working directory + Windows path strings

- Print working directory when debugging:

```
System.out.println(new java.io.File(".").
    getAbsolutePath());
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Working directory + Windows path strings

- Print working directory when debugging:

```
System.out.println(new java.io.File(".").
    getAbsolutePath());
```

- Windows paths in Java strings need escaping:

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Working directory + Windows path strings

- Print working directory when debugging:

```
System.out.println(new java.io.File(".").
    getAbsolutePath());
```

- Windows paths in Java strings need escaping:
  - wrong: `"D:\data\marks.txt"`

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Working directory + Windows path strings

- Print working directory when debugging:

```
System.out.println(new java.io.File(".").
    getAbsolutePath());
```

- Windows paths in Java strings need escaping:
  - wrong: `"D:\data\marks.txt"`
  - correct: `"D:\\data\\marks.txt"` or `"D:/data/marks.txt"`

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint and Reflection

Reflection prompt

Where would *Java I/O Streams + FileReader/FileWriter* matter most in Campus Resource Manager, and which design choice would you justify using this lecture's ideas?

Quick self-check

Which part needs one more example before you move on: *Stream model (big picture)*, *Byte streams vs character streams*, the worked example, or the recap?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example (1/2): read + compute (safe I/O)

```
import java.io.*;

public class MarksReport {
  public static void main(String[] args) {
    try (BufferedReader br = new BufferedReader(new FileReader("
        marks.txt"));
         BufferedWriter bw = new BufferedWriter(new FileWriter("
             report.txt"))) {

      int sum = 0, max = 0, count = 0;
      String line;
      while ((line = br.readLine()) != null) {
        line = line.trim();
        if (line.isEmpty()) continue;
        int x = Integer.parseInt(line);
        if (count == 0 || x > max) max = x;
        sum += x; count++;
      }
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., Thread, AutoCloseable) (accessed April 1, 2026).

Example (2/2): write report + handle errors

```
      double avg = (count == 0) ? 0.0 : (sum / (double) count);
      bw.write("Count=" + count); bw.newLine();
      bw.write("Max=" + (count == 0 ? "N/A" : max)); bw.newLine
          ();
      bw.write(String.format("Avg=%.2f", avg)); bw.newLine();
    }
    catch (IOException e) {
      System.out.println("I/O error: " + e.getMessage());
    }
    catch (NumberFormatException e) {
      System.out.println("Bad number in file: " + e.getMessage()
          );
    }
  }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Text file: byte stream or character stream?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Text file: byte stream or character stream?
2. Why is buffering useful?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Text file: byte stream or character stream?
2. Why is buffering useful?
3. What does `try-with-resources` guarantee?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Watch-outs

- Forgetting to close streams or readers.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Watch-outs

- Forgetting to close streams or readers.
- Using byte streams when character streams fit text better.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Watch-outs

- Forgetting to close streams or readers.
- Using byte streams when character streams fit text better.
- Reporting vague file errors with no recovery hint.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Takeaways

- Streams model I/O; choose byte vs character based on data type.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Takeaways

- Streams model I/O; choose byte vs character based on data type.
- Use buffering for efficiency and line-based reading.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Takeaways

- Streams model I/O; choose byte vs character based on data type.
- Use buffering for efficiency and line-based reading.
- Use `try-with-resources` for safe closing and cleaner code.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint Questions

1. When do you choose `InputStream` over `Reader`?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint Questions

1. When do you choose `InputStream` over `Reader`?
2. Why do we prefer `BufferedReader` for reading lines?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint Questions

1. When do you choose `InputStream` over `Reader`?
2. Why do we prefer `BufferedReader` for reading lines?
3. What is the working directory and why does it matter for file paths?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.
- Apply the idea once inside Campus Resource Manager to make the concept stick.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Unit Preview

Next: Unit 04 – Generics Fundamentals (Generic Classes)

- We will learn type-safe reusable code using <T>.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).